

QUESTIONS:

1. En general, avant YOLO, les méthodes de détection d'objets — comme **R-CNN**, **Fast R-CNN** — fonctionnaient en **deux temps**:

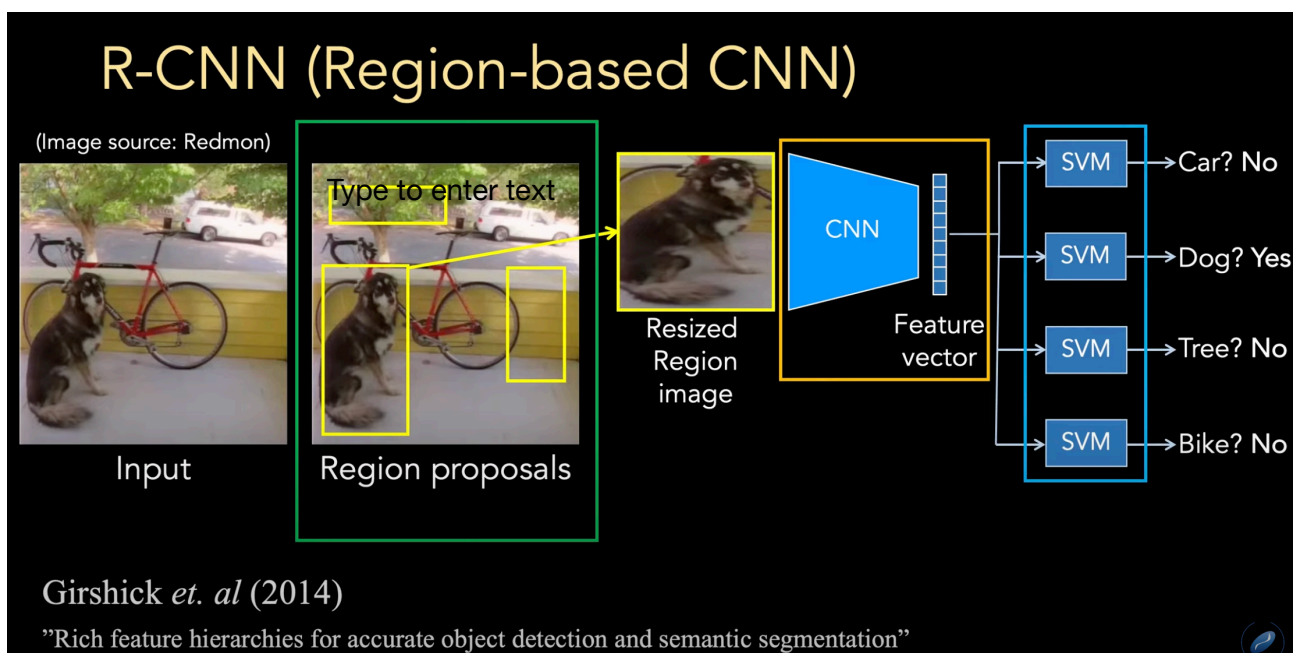
1. **Proposition de régions (Region Proposal)**

→ Le réseau recherche des zones qui pourraient contenir des objets (souvent plusieurs milliers de régions).

2. **Classification pour chaque région**

→ Chaque zone candidate est ensuite **recadrée** et envoyée dans un réseau CNN pour savoir :

- s'il y a un objet ou non,
- et si oui, quelle classe (chien, voiture, personne...).



Au lieu de traiter la détection comme plusieurs classifications, **YOLO la traite comme une régression directe à partir de l'image entière.**

Classification = prédire une *catégorie discrète* (chien, chat, voiture)

Régression = prédire des *valeurs continues* (coordonnées, tailles, probabilités, etc.)

YOLO: il apprend une fonction unique qui mappe directement l'image vers **toutes les coordonnées et les classes d'objets.**

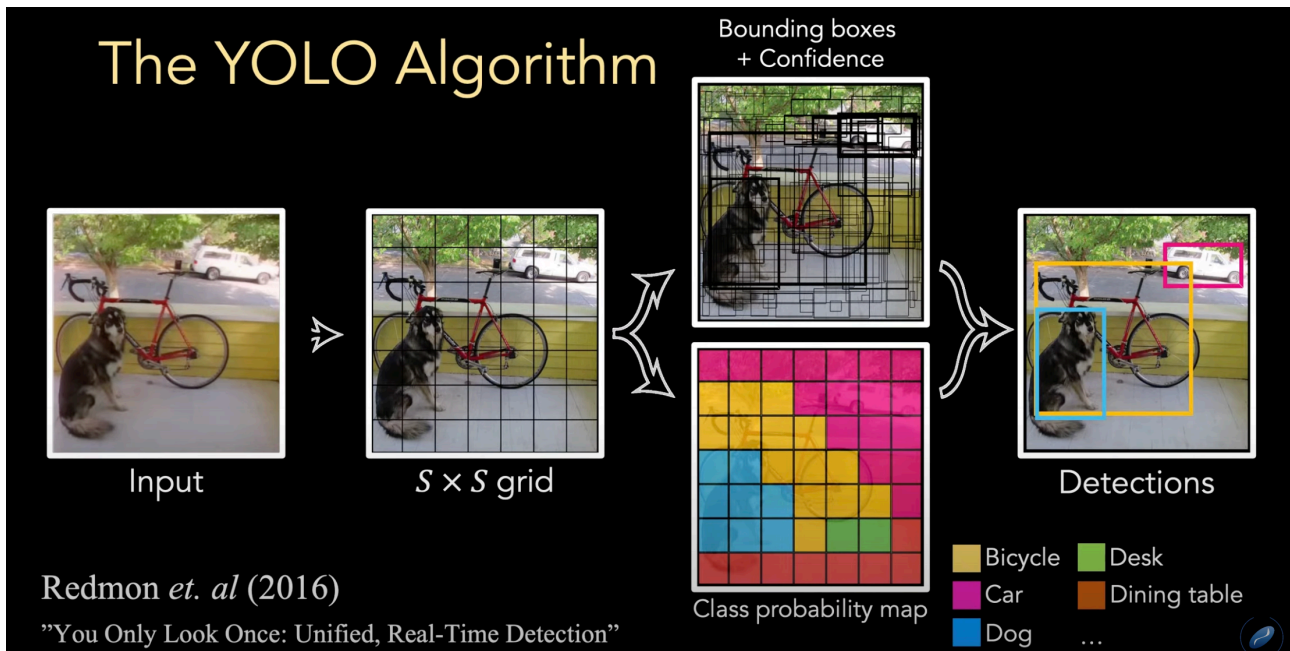
$$f(\text{image}) \rightarrow (x, y, w, h, \text{confiance}, \text{classes})$$

YOLO produit une sortie $7 \times 7 \times 30$ (pour YOLOv1),
et cette sortie contient **toutes les informations nécessaires** :

- les coordonnées des boîtes (x, y, w, h),
- la probabilité qu'une boîte contienne un objet,
- et les probabilités de chaque classe.

Le réseau est entraîné *de bout en bout* (end-to-end) à minimiser **une seule erreur globale**
 $L_{tot} = L_{localisation} + L_{confiance} + L_{classe}$ — et cette optimisation ajuste tous les poids pour que le réseau apprenne **à la fois** :

- où sont les objets,



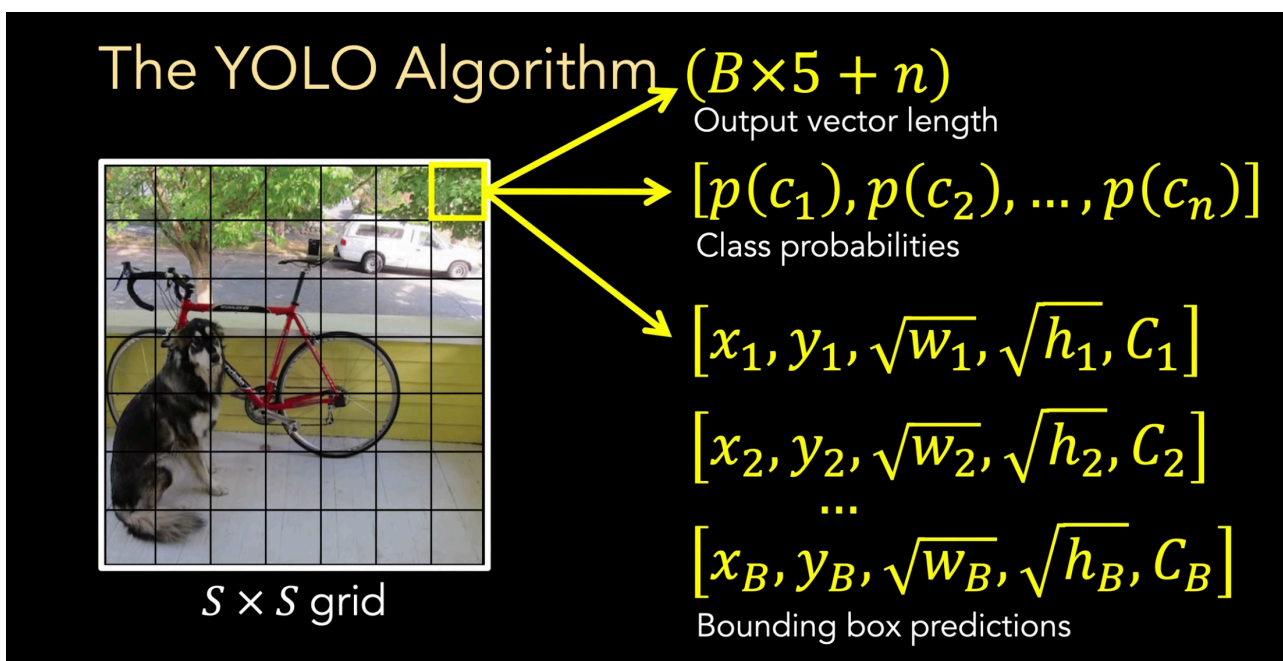
- et quels objets c'est.

On donne à YOLO une **image d'entraînement** (par exemple 448x448 pixels) et les **annotations réelles** correspondantes, c'est-à-dire :

- les **boîtes englobantes** de chaque objet (x, y, w, h)
- la **classe** de chaque objet (ex : « chien », « personne », « voiture », etc.)

Exemple :

Objet	x	y	w	h	Classe
Chien	0.45	0.60	0.30	0.25	"chien"

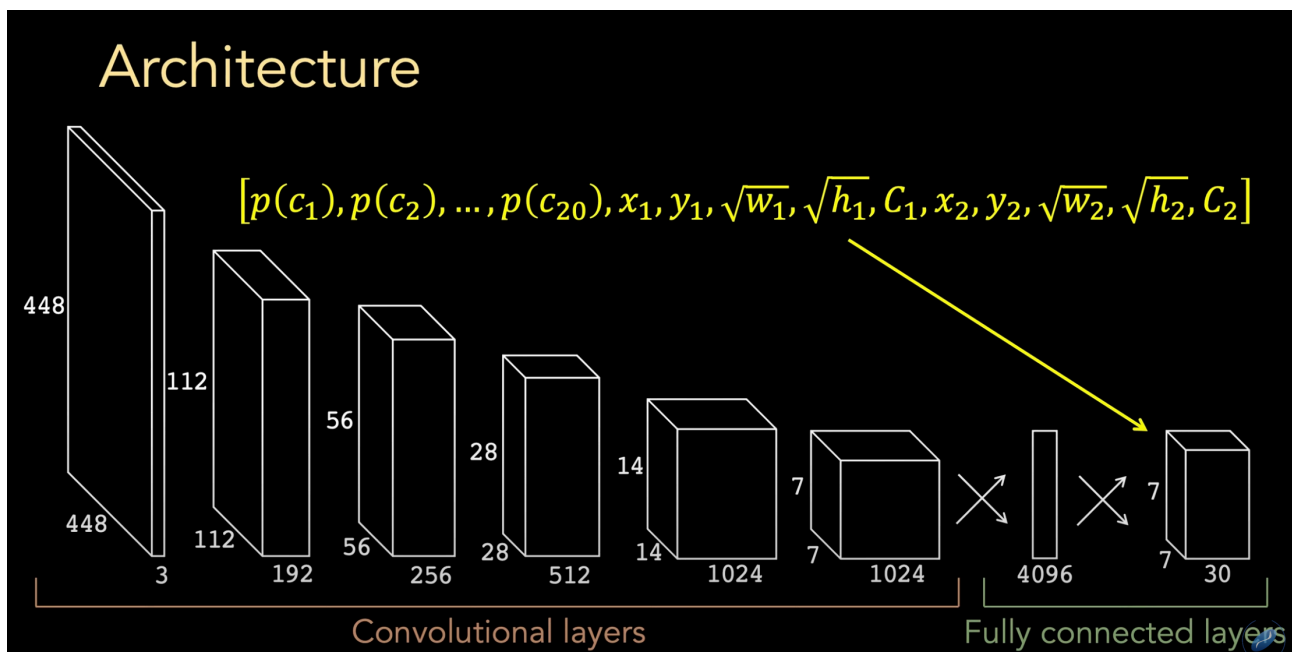


NB: Les coordonnées sont **normalisées** entre 0 et 1 par rapport à la taille de l'image.

L'image est envoyée dans le **réseau convolutionnel (CNN)** de YOLO, qui extrait des features (les plus importants) (bords, textures, formes, etc.).

En sortie, YOLO produit un **tenseur 7×7×30** (YOLOv1), qui représente :

- pour chaque cellule de la grille (7×7, paramètre à fixer),
- **2 boîtes** possibles (donc 2×5 valeurs)
- et **20 probabilités de classes**



Au total : 49 cellules × 30 valeurs (**20** class probabilities + **4** coordonnées et **1** pour la classe dans la boîtes, pour **2** Boîtes) = **1470 prédictions** pour une seule image.

C1 et C2 dans l'image sont la CLASSE dans les boîtes.

La question de YOLO sera: Quelle cellule et quelle boîte prédite vont apprendre pour chaque objet réel ?

1. On regarde **où se trouve le centre de chaque objet réel**.
→ La cellule de la grille qui contient ce centre devient responsable de cet objet.
2. Parmi les 2 boîtes prédites par cette cellule,
→ celle qui a la **plus grande IoU (Intersection over Union)** avec la boîte réelle est choisie.
→ Cette boîte devient la « responsable » de cet objet.

L'autre boîte ne sera pas modifiée pour cet objet-là.

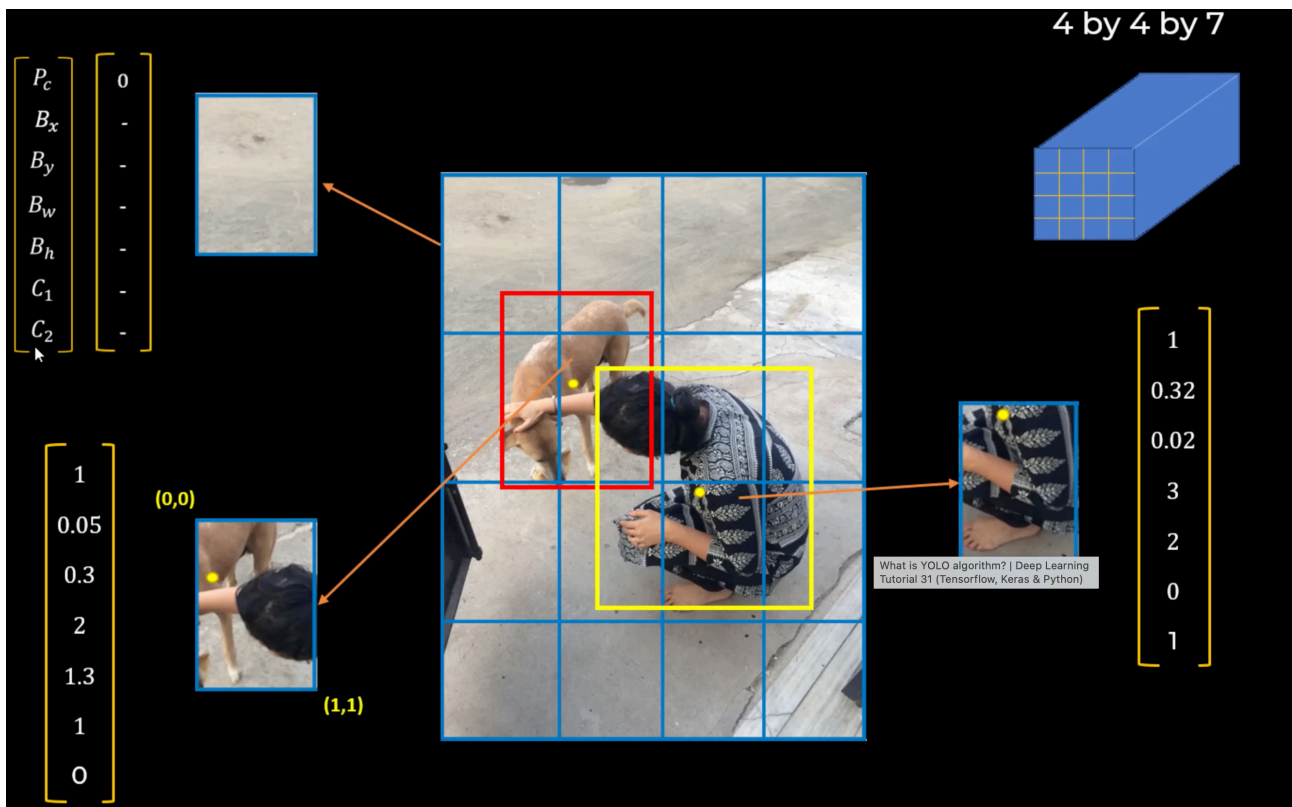


Fig.3

Example Fig.3: L'image suivante montre une image utilisée pour l'entraînement avec les annotations correspondantes. X_train et Y_train respectivement. Les points jaunes sont les centres des objets (2 classes de sortie C1 et C2). Dans cet exemple, la grille est de 4x4 et chaque cellule a un vecteur de 7 (2 classes + 5 coordonnées pour une boîte B).

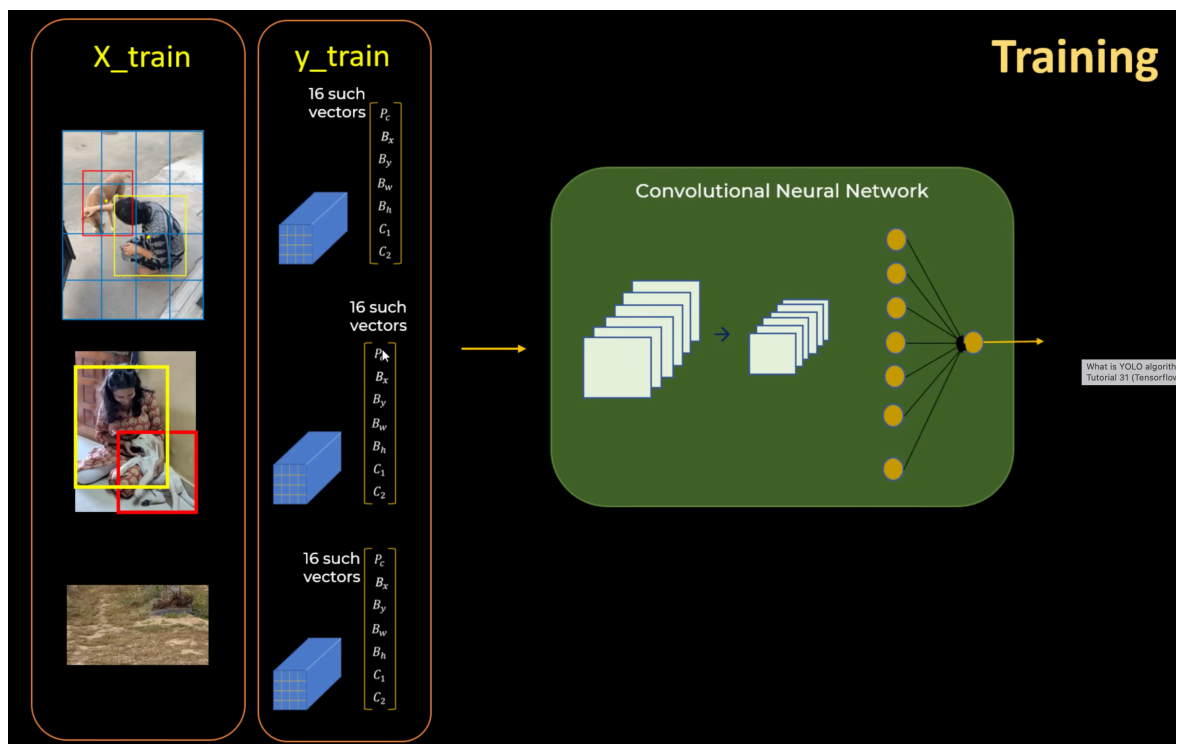


Fig.4

Comme le montre la figure 4, X_{train} sera composé de nombreuses images accompagnées des annotations correspondantes qui serviront à entraîner le modèle. En particulier, y_{train} , c'est-à-dire les annotations ou vérités terrai , sera utilisé par YOLO pour comparer ses prédictions à l'aide d'une **fonction de perte composée de trois parties** :

$$\text{Loss} = \lambda_{\text{coord}} \left[(x - \hat{x})^2 + (y - \hat{y})^2 + (\sqrt{w} - \sqrt{\hat{w}})^2 + (\sqrt{h} - \sqrt{\hat{h}})^2 \right] + (C - \hat{C})^2 + \lambda_{\text{noobj}}(C - \hat{C})^2 + \sum_{c=1}^C (p(c) - \hat{p}(c))^2$$

Perte de localisation

**Perte de confiance
(Object ou non)**

**Perte de
classification**

Perte de localisation: Compare les coordonnées (x, y, w, h) prédites à la vérité;

Perte de confiance: Compare le score de confiance à la réalité (objet ou pas);

Perte de classification: Compare les classes prédites à la classe réelle.

Puis il **met à jour ses poids** avec la Loss (paramètres internes) pour **réduire la perte** la prochaine fois.

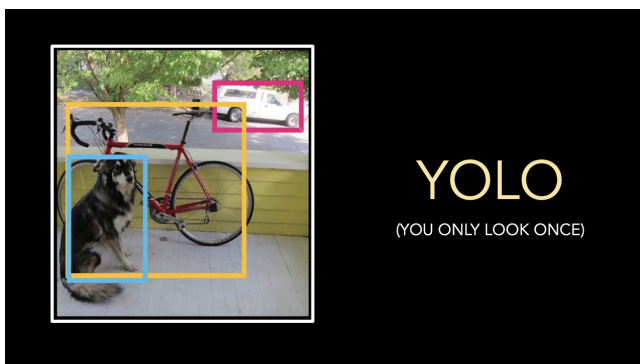
Le processus est répété pour **toutes les images du jeu d'entraînement** :

- Chaque image → sortie 7×7×30 1er exemple ou 4x4x7 pour le deuxième exemple;
- Calcul de la perte;
- Mise à jour des poids.

YOLO apprend à :

- Positionner correctement les boîtes (2 dans le premier exemple et 1 boîte dans le deuxième);
- Identifier les classes;
- Être confiant quand un objet est présent.

VIDEO TRES UTILE:



2.

If you need real-time detection (e.g. in robotics or live video), DPM's speed might justify its inaccuracy.

If accuracy matters more (e.g. in medical or autonomous-driving tasks), slower but more precise models are better.

Le **Deformable Part Model (DPM)** repose sur un paradigme *classique* de la vision par ordinateur. Il utilise des **caractéristiques manuellement conçues** (comme les HOG — *Histogram of Oriented Gradients*) et un modèle linéaire pour détecter les objets. Ces caractéristiques capturent essentiellement des **bords et des formes locales**, mais elles ne permettent pas de représenter la **variabilité complexe** des objets réels (éclairage, texture, déformation, point de vue, etc.).

Les modèles modernes comme **Fast R-CNN** et **YOLO** s'appuient sur des **réseaux de neurones convolutifs (CNN)** capables d'**apprendre automatiquement** des représentations hiérarchiques des images.

Ces réseaux apprennent directement à partir des données quelles caractéristiques sont pertinentes pour reconnaître et localiser les objets.

Ainsi, ils sont **beaucoup plus discriminants et plus robustes** aux variations visuelles.

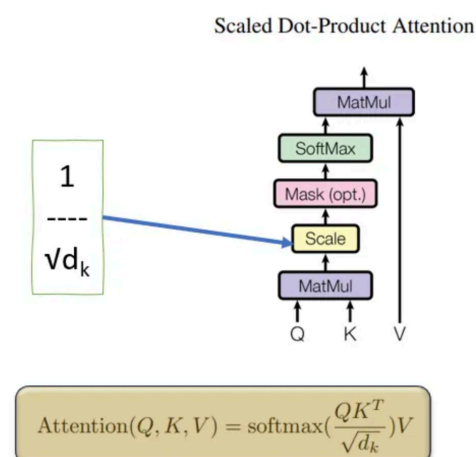
DPM est beaucoup plus rapide que Fast-CNN et il est presque à YOLO, cela parce que **DPM effectue uniquement des opérations linéaires** (produits de convolution entre les filtres HOG et l'image), qui sont très rapides à calculer. Il n'implique pas de réseau profond ni de calculs intensifs sur GPU. Son pipeline est **entièrement déterministe et peu coûteux en mémoire**.

Fast R-CNN et YOLO reposent sur des **réseaux profonds** comprenant **des millions de paramètres** et des **calculs convolutifs lourds**, ce qui augmente fortement le temps de traitement par image

TRANSFORMER

1.

Le mécanisme d'attention permet à un modèle de **focaliser son attention sur certaines parties de la séquence d'entrée** lors de la prédiction d'une sortie.



- Dans une phrase, certains mots sont plus importants pour prédire le mot suivant.
- Le mécanisme d'attention attribue un **poids** à chaque élément de la séquence, indiquant son **importance relative**.

Soit une séquence d'entrées transformée en **vecteurs**. Pour chaque élément de la séquence, on calcule trois vecteurs :

1. **Query (Q)** – ce que l'on cherche.
2. **Key (K)** – ce que chaque élément contient.
3. **Value (V)** – l'information réelle associée à chaque élément.

Le mécanisme d'attention se résume à trois étapes :

Étape 1 : Calcul des scores d'attention

Pour chaque **Query**, on calcule la **similarité avec tous les Keys** :

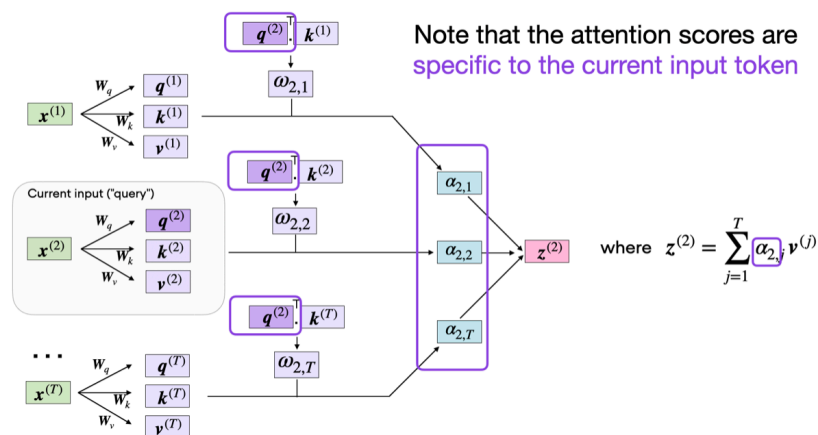
1. $\text{score}(Q, K) = QK^T$, où $Q \in R^{d_k}$ est la Query et $K \in R^{d_k}$ est la Key.

2. $\alpha_i = \frac{\exp\left(\frac{\text{score}_i}{\sqrt{d_k}}\right)}{\sum_j \exp\left(\frac{\text{score}_j}{\sqrt{d_k}}\right)}$, α_i est le poids d'attention pour le i-ème élément. d_k sert à stabiliser les gradients lors de l'apprentissage.

3. $\text{Attention}(Q, K, V) = \sum_i \alpha_i V_i$, V_i est le vecteur Value associé à la Key K_i . La sortie est une **moyenne pondérée des Values**, pondérée par les poids d'attention α_i .

4. $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Chaque élément **regarde tous les autres éléments** pour décider à **qui prêter attention**.



Ce site c'est bien fait: <https://sebastianraschka.com/blog/2023/self-attention-from-scratch.html>.

2.

Self-Attention

Principe :

Les **Queries (Q)**, **Keys (K)** et **Values (V)** proviennent tous de la même source.

Elle mesure comment chaque élément d'une séquence se rapporte aux autres éléments de la même séquence.

Exemple:

Phrase : *"Le chat noir dort sur le tapis."*

- Chaque mot "attend" les autres mots pour enrichir sa représentation.
- Exemple : Le mot "**dort**" porte beaucoup d'attention sur "**chat**", car il indique qui dort.

Process mathématique :

1. On prend la même séquence d'entrée et on crée trois matrices : **Q**, **K**, **V**.
2. On multiplie $\mathbf{Q} \times \mathbf{K}^T$, puis on applique **softmax** → cela donne les **scores d'attention**.
3. On multiplie ces scores par **V** → cela produit la sortie finale de l'attention.

Clé : Tout vient de la même séquence. Les représentations des mots sont enrichies par le contexte complet.

Cross-Attention

Principe :

- Les **Queries (Q)** viennent d'une source (ex. le décodeur).
- Les **Keys (K)** et **Values (V)** viennent d'une autre source (ex. l'encodeur).
Elle mesure comment les éléments d'une séquence se rapportent aux éléments d'une autre séquence.

Exemple:

Dans la traduction automatique :

- Le décodeur (français) produit **Q** à partir de ses états internes (hidden states).
- L'encodeur (anglais) fournit **K** et **V** à partir de sa sortie.
- Le décodeur regarde les sorties de l'encodeur pour décider quels mots anglais sont pertinents pour générer chaque mot français.

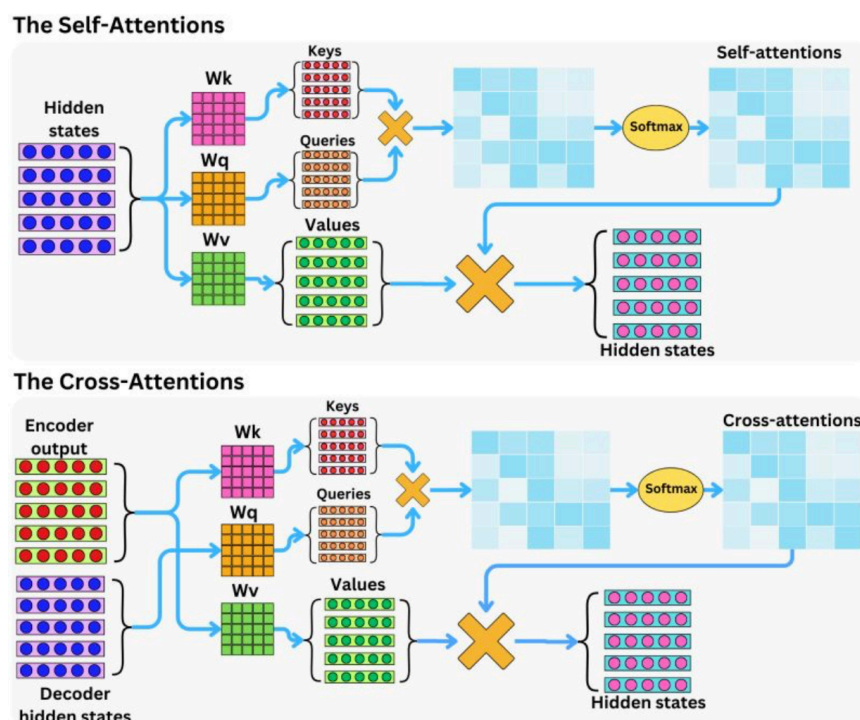
Process mathématique :

1. Q provient du décodeur, K et V de l'encodeur.
2. On multiplie $Q \times K^T$, puis on applique **softmax** $\rightarrow$ scores d'attention.
3. On multiplie ces scores par $V \rightarrow$ sortie finale.

Clé : Le décodeur regarde la séquence source pour s'aider à générer la séquence cible.

—> **Résumé du calcul pour les deux attentions :**

- On projette les entrées via trois matrices W_q, W_k, W_v pour obtenir Q, K, V .
- **Attention** = $\text{softmax}(Q \times K^T) \times V$.
- La différence est uniquement la source des vecteurs Q, K, V .



3.

Pour donner au modèle un sens de l'ordre des mots, on ajoute **des vecteurs de position** aux vecteurs d'embeddings de mots.

ORDRE DANS LES TRANSFORMERS

- Chaque mot x_i est transformé en un vecteur d'embedding $E(x_i)$.
- On ajoute à ce vecteur un **embedding de position** $P(i)$:

Input final = $E(x_i) + P(i)$

- Donc, chaque mot "sait" où il se trouve dans la séquence.

TYPES DE POSITION ENCODINGS

1. Position encodings sinusoïdaux (ex: rotary positional embeddings ROPE): Pour chaque dimension du vecteur, on utilise des fonctions sin et cos de fréquences différentes.

Avantage: permet au modèle de généraliser à des longueurs de séquence jamais vues.

2. Position embeddings appris: chaque position a un vecteur qui est appris directement pendant l'entraînement.

Avantage: le modèle peut apprendre des patterns spécifiques à la tâche, mais limité à la longueur maximale vue pendant l'entraînement.

RECONNAISSANCE DES FRONTIÈRES DE PHRASE

- Pour signaler **le début ou la fin d'une séquence**, on ajoute souvent des tokens spéciaux :
 - [CLS] : token de classification (début)
 - [SEP] : token de séparation / fin de séquence